Customer Churn Analysis & Revenue Forecast

Python project: XGBoost churn model, risk scoring and 12-month revenue forecast for managed IT service customers.

1. Source Code (churn_analysis.py)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.pipeline import make_pipeline
from sklearn.metrics import classification_report, roc_auc_score
from xgboost import XGBClassifier
from statsmodels.tsa.holtwinters import ExponentialSmoothing
import os

# Load the company data
df = pd.read_csv("managed_it_transactions.csv")

os.makedirs("outputs", exist_ok=True)

# Check the data before starting
print("Dataset shape:", df.shape)
print(df.head())
print("\nMissing values:")
print(df.isnull().sum())

# Convert dates into the correct format
df["transaction_date"] = pd.to_datetime(
    df["transaction_date"], errors="coerce"
)

# Create one record for each customer
customers = df.groupby("customer_id").agg({
    "revenue": ["sum", "mean"],
    "usage_hours": "mean",
    "ticket_count": "mean",
    "sla_breaches": "sum",
    "transaction_date": ["max", "nunique"],
    "churned": "max"
})

customers.columns = [
    "_".join(col).strip("_") for col in customers.columns
]
customers = customers.reset_index()

customers.rename(columns={
    "revenue_sum": "total_revenue",
    "revenue_mean": "average_revenue",
    "usage_hours_mean": "average_usage",
    "ticket_count_mean": "average_tickets",
    "sla_breaches_sum": "total_sla_breaches",
    "transaction_date_max": "last_transaction",
    "transaction_date_nunique": "active_periods",
    "churned_max": "churned"
}, inplace=True)

# Calculate how long each customer has been inactive
last_date = df["transaction_date"].max()

customers["days_inactive"] = (
    last_date - customers["last_transaction"]
).dt.days

# Prepare the data for the churn model
features = [
    "total_revenue",
    "average_revenue",
    "average_usage",
    "average_tickets",
    "total_sla_breaches",
    "active_periods",
```

```
    "days_inactive"
]

customers = customers.dropna(subset=["churned"])
customers = customers[customers["churned"].isin([0, 1])]

X = customers[features].replace([np.inf, -np.inf], np.nan)
y = customers["churned"].astype(int)

# Keep both churned and retained customers in each split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Fill missing feature values using the median
positive = y_train.sum()
negative = len(y_train) - positive

model = make_pipeline(
    SimpleImputer(strategy="median"),
    XGBClassifier(
        n_estimators=200,
        max_depth=4,
        learning_rate=0.05,
        scale_pos_weight=negative / max(positive, 1),
        eval_metric="logloss",
        random_state=42,
        n_jobs=-1
    )
)

model.fit(X_train, y_train)

# Check how well the model performs
probabilities = model.predict_proba(X_test)[:, 1]

print("\nModel ROC-AUC:", round(
    roc_auc_score(y_test, probabilities), 3
))

print(classification_report(
    y_test,
    (probabilities >= 0.5).astype(int),
    zero_division=0
))

# Find customers who may leave the company
customers["churn_probability"] = model.predict_proba(X)[:, 1]
customers["risk_level"] = np.where(
    customers["churn_probability"] >= 0.6,
    "High",
    np.where(
        customers["churn_probability"] >= 0.3,
        "Medium",
        "Low"
    )
)

# Estimate the revenue exposed to churn risk
customers["revenue_at_risk"] = (
    customers["total_revenue"].fillna(0).clip(lower=0)
    * customers["churn_probability"]
)

# Suggest an action for each customer
def suggest_action(row):
    if row["risk_level"] == "High":
        return "Contact customer and review service issues"

    if row["total_sla_breaches"] > 0:
        return "Check SLA performance"

    if row["days_inactive"] > 60:
        return "Arrange a customer follow-up"

    return "Continue regular customer engagement"
```

```
customers["suggested_action"] = customers.apply(
    suggest_action, axis=1
)

customers.sort_values(
    "churn_probability", ascending=False
).to_csv("outputs/customer_churn_report.csv", index=False)

# Study the main factors behind churn predictions
imputer = model.named_steps["simpleimputer"]
classifier = model.named_steps["xgbclassifier"]

importance = pd.DataFrame({
    "factor": features,
    "importance": classifier.feature_importances_
}).sort_values("importance", ascending=False)

print("\nMain churn factors:")
print(importance.to_string(index=False))

importance.to_csv("outputs/churn_factors.csv", index=False)

# Forecast revenue for the next 12 months
monthly = (
    df.dropna(subset=["transaction_date"])
      .groupby(pd.Grouper(key="transaction_date", freq="MS"))["revenue"]
      .sum(min_count=1)
      .sort_index()
)
monthly = monthly.interpolate(
    method="time"
).ffill().bfill()

if len(monthly) < 12:
    raise ValueError(
        "At least 12 months of revenue history are needed."
    )

if len(monthly) >= 24:
    forecast_model = ExponentialSmoothing(
        monthly,
        trend="add",
        seasonal="add",
        seasonal_periods=12
    )
else:
    forecast_model = ExponentialSmoothing(
        monthly,
        trend="add"
    )

forecast = forecast_model.fit().forecast(12).clip(lower=0)

forecast_data = pd.DataFrame({
    "month": forecast.index,
    "predicted_revenue": forecast.values
})

forecast_data.to_csv(
    "outputs/revenue_forecast.csv", index=False
)

# Plot historical and predicted revenue
plt.figure(figsize=(10, 5))
plt.plot(monthly.index, monthly.values, label="Actual revenue")
plt.plot(forecast.index, forecast.values, label="Forecast")
plt.title("Monthly Revenue Forecast")
plt.xlabel("Month")
plt.ylabel("Revenue")
plt.legend()
plt.tight_layout()
plt.savefig("outputs/revenue_forecast.png")
plt.show()

print("\nHigh-risk customers:",
      (customers["risk_level"] == "High").sum())

print("Expected revenue at risk:",
      round(customers["revenue_at_risk"].sum(), 2))
```

```
print("\nReports saved in the outputs folder.")
```

2. Output

```
PS C:\Users\kesha\Downloads\churn.project> python churn_analysis.py


Main churn factors:
            factor  importance
   average_tickets         1.0
   average_revenue         0.0
     total_revenue         0.0
     average_usage         0.0
total_sla_breaches         0.0
    active_periods         0.0
     days_inactive         0.0
```